TP5 Radar – Guía del código

Sistemas de Radiocomunicación – FCEFyN, UNC Script: `Actividad_5/practico/tp5_radar.py`

Esta guía recorre los puntos del código que pide la consigna. Para cada uno muestra el fragmento del script, explica qué hace y da la justificación teórica. Al principio está el paso a paso para correrlo y reproducir los resultados de los informes. Los números de línea son los del archivo.

Índice

1. Paso a paso para reproducir los resultados **1**

1.1. Requisitos . . . 1

1.2. Opción A: desde una terminal (recomendada) . . . 2

1.3. Opción B: con Anaconda y Spyder (sugerida por la consigna) . . . 2

1.4. Resultados esperados . . . 2

1.5. Dónde está cada punto de la consigna . . . 3

2. Lectura de la señal: tiempo lento y tiempo rápido **3**

3. Parámetros derivados y del CFAR **3**

4. Compresión con filtro apareado **4**

4.1. El chirp y el filtro apareado . . . 4

4.2. Convolución por FFT y recorte . . . 4

5. Ventana CFAR y umbral **5**

6. Detección por cruces por cero **6**

7. Canceladores MTI simple, STI simple y MTI doble **7**

8. CFAR plot de cada cancelador **7**

9. Filtrado Doppler **8**

9.1. Detección sobre el mapa rango-velocidad . . . 9

10. Clasificación de los blancos **9**

11. Gráficos que pide la consigna **10**

1. Paso a paso para reproducir los resultados

1.1. Requisitos

- Python 3.10 o posterior (se probó con Python 3.14).
- Bibliotecas: `numpy`, `scipy`, `pandas` y `matplotlib`.
- Los archivos `signal_1.csv`, `signal_2.csv` y `signal_example.csv`, en la misma carpeta que el script (`Actividad_5/practico/`).

1.2. Opción A: desde una terminal (recomendada)

1. Instalar las bibliotecas (una sola vez):

```
pip install numpy scipy pandas matplotlib
```

2. Ir a la carpeta del script:

```
cd Actividad_5/practico
```

3. Validar primero con la señal de ejemplo de la consigna:

```
python tp5_radar.py example
```

   Tiene que detectar tres blancos móviles en 10,02, 15,02 y 20,01 km (la Figura 5 de la consigna los muestra en 9,95, 14,97 y 19,93 km) y uno fijo en 18,02 km.

4. Procesar la señal de cada grupo:

```
python tp5_radar.py 1
python tp5_radar.py 2
```

5. Revisar la salida. El script imprime los parámetros, los rangos detectados por cada cancelador y la lista final de blancos, y guarda en `Actividad_5/Figuras/senal_<n>/`:

   - `01_senal_recibida.png` a `08_mapa_rango_velocidad.png`: los gráficos que pide la consigna.
   - `resultados.json`: todas las detecciones, las descartadas con su motivo y los blancos finales.

   Cada corrida tarda unos segundos y sobrescribe las figuras de esa señal.

1.3. Opción B: con Anaconda y Spyder (sugerida por la consigna)

1. Abrir `tp5_radar.py` en Spyder. Anaconda ya trae las cuatro bibliotecas.
2. Para elegir la señal: *Run* → *Configuration per file* → *Command line options*, y escribir `1`, `2` o `example`. Sin argumento procesa la señal 1.
3. Ejecutar con F5. Las celdas `#%%` también se pueden correr de a una con Ctrl+Enter para ver cada etapa.
4. Las figuras se guardan como PNG en la carpeta `Figuras`; el script no abre ventanas.

1.4. Resultados esperados

Señal	Blancos móviles [km; m/s]	Blancos fijos [km]
1	10,02 (+85,7); 15,21 (−40,2); 15,51 (+22,7); 20,01 (+43,7); 22,23 (+31,5)	12,02; 17,61; 21,02
2	8,01 (+31,5); 12,02 (+59,4); 19,82 (+24,5); 20,22 (+43,7); 26,24 (−85,7)	15,62; 20,31; 22,02
example	10,02 (+50,7); 15,02 (−40,2); 20,01 (+29,7)	18,02

Si los números coinciden, el procesamiento está reproducido. Velocidad positiva: el blanco se acerca; negativa: se aleja.

1.5. Dónde está cada punto de la consigna

Consigna	Código	Figura
Compresión con filtro apareado	`matched_filter`, `comprimir()`	`02_compresion`
Ventana CFAR	`ventana_cfar()`, `umbral_cfar()`	`03_ventana_cfar`
MTI simple, STI simple, MTI doble	`mti_sc`, `sti_sc`, `mti_dc`	04, 05, 06
CFAR plot y cruces por cero	`detectar()`, `cfar_plot()`	04 a 06
Filtro Doppler	`FD`, `doppler`	`07_doppler_3d`, 08
Señal recibida (un momento de muestreo)	Plot 1	`01_senal_recibida`

2. Lectura de la señal: tiempo lento y tiempo rápido

```
radar_signal = pd.read_csv(f'signal_{SENAL}.csv', index_col=None)
radar_signal = np.array(radar_signal['real'] + 1j*radar_signal['imag'])
radar_signal = radar_signal.reshape(Np, -1)        # filas: tiempo lento (pulsos); columnas:
    tiempo rápido (rango)
```

Qué hace. El CSV trae 200.000 muestras en dos columnas, la parte real y la imaginaria. Se arman como números complejos y se ordenan en una matriz de 100×2000: cada fila es un pulso y cada columna, una distancia.

Justificación teórica. El receptor entrega las componentes en fase y en cuadratura (I/Q) de la señal en banda base. Se necesitan las dos porque la fase del eco lleva la información de velocidad. Después de cada pulso transmitido, el receptor muestrea el eco a $f_s = 10$ MHz:

- **Tiempo rápido** (columnas): las muestras dentro de un mismo pulso. La muestra n llega nt_s después de transmitir, que corresponde a la distancia $R = cnt_s/2$. El paso en rango es $c/(2f_s) = 15$ m, y 2000 muestras cubren 30 km.
- **Tiempo lento** (filas): el mismo rango de un pulso al siguiente, separados una PRI $= 1/PRF =$ 666,7 μs. A lo largo de las filas se ve cómo cambia la fase de un blanco que se mueve.

El procesamiento en rango (compresión, CFAR) se hace a lo largo de las columnas, y el procesamiento de velocidad (canceladores, Doppler) a lo largo de las filas.

3. Parámetros derivados y del CFAR

```
wlen = c/fc                          # Wavelength [m]
PRI = PRF**(-1)                      # Pulse repetition interval [s]
ru = (c*(PRI-Tp-Te))/2               # Unambigous Range [m]
vu_ms = wlen*PRF/2                   # Unambigous Velocity [m/s]
rank_min = (Tp/2+Te)*c/2             # Minimum Range [m]
rank_max = 30e3                      # Maximum Range [m]
rank_res = ts*c/2                    # Range Step [m]
tmax = 2*rank_max/c                  # Maximum Simulation Time
res_rango = c/(2*BW)                 # Resolución en rango del chirp comprimido [m]

# Parámetros del CFAR (CA-CFAR)
N_REF = 64              # celdas de referencia (cómputo), 32 de cada lado
N_GAP = 8               # celdas de guarda de cada lado de la celda bajo prueba
PFA = 1e-6              # probabilidad de falsa alarma de diseño
ALFA = N_REF*(PFA**(-1/N_REF) - 1)   # factor del umbral en potencia (detector cuadrático)
GANANCIA = np.sqrt(ALFA)             # factor del umbral en amplitud
```

Qué hace. Calcula las magnitudes que limitan al radar y fija los parámetros del detector CFAR.

Justificación teórica.

- $\lambda = c/f_c = 23{,}1$ cm. Con ella se convierte frecuencia Doppler en velocidad.
- **Rango no ambiguo** $R_u = c\,(PRI - T_p - T_e)/2 = 97{,}75$ km: un eco más lejano llegaría después del pulso siguiente y se confundiría con uno cercano.
- **Velocidad no ambigua** $v_u = \lambda\, PRF/2 = 173$ m/s (`vu_ms`): es el ancho total del intervalo de velocidades que se pueden medir. La DFT en tiempo lento solo distingue frecuencias Doppler entre $-PRF/2$ y $+PRF/2$, es decir velocidades entre $\pm\lambda\, PRF/4 = \pm 86{,}5$ m/s. Una más rápida aparece replegada (aliasing).
- **Rango mínimo** $(T_p/2 + T_e)\, c/2 = 1{,}5$ km: mientras se transmite y el receptor se recupera, el duplexor lo mantiene bloqueado.
- **Resolución en rango** $c/(2B) = 75$ m: la que da el chirp comprimido (sección 4).
- **Factor del CFAR** $\alpha = N(P_{fa}^{-1/N} - 1) = 15{,}4$ y su raíz 3,93 (sección 5).

4. Compresión con filtro apareado

4.1. El chirp y el filtro apareado

```
Npts = int(tmax/ts)                                  # Simulation Points
t = np.linspace(-tmax/2, tmax/2, Npts)
ranks = np.linspace(rank_res, rank_max, Npts)        # Range Vector

tx_chirp = np.exp(1j*np.pi*BW/Tp * t**2)             # Tx Linear Chirp (t)
tx_rect = np.where(np.abs(t) <= Tp/2, 1, 0)          # Rect Function
tx_chirp = tx_rect*tx_chirp                          # Tx Chirp Rectangular

# Matched Filter: h(t) = s*(-t)
matched_filter = np.conj(np.flip(tx_chirp))
```

Qué hace. Genera el chirp que transmite el radar (10 μs, 2 MHz) y, a partir de él, el filtro apareado: el mismo chirp invertido en el tiempo (`flip`) y conjugado (`conj`).

Justificación teórica. El chirp es un pulso con modulación lineal de frecuencia:

$$s(t) = \mathrm{rect}(t/T)\, e^{j\pi(B/T)t^2}, \qquad f_i(t) = \frac{1}{2\pi}\frac{d\phi}{dt} = \frac{B}{T}\,t$$

La fase es cuadrática, así que la frecuencia instantánea recorre linealmente el ancho de banda B. El filtro que maximiza la relación señal a ruido en ruido blanco es el apareado, $h(t) = s^*(-t)$: por la desigualdad de Schwarz la SNR de salida es como máximo $2E/N_0$, y la igualdad se alcanza con $H(f) = S^*(f)$. En frecuencia, $S^*(f)$ cancela la fase cuadrática del chirp: todas sus componentes salen alineadas en el mismo instante y forman un pico angosto.

4.2. Convolución por FFT y recorte

```
def fastconv(A, B):
    """Convolución rápida por FFT (función provista por la cátedra)."""
    out_len = len(A) + len(B) - 1
    sizefft = int(2**(np.ceil(np.log2(out_len))))      # potencia de 2 más próxima
    Afilled = np.concatenate((A, np.zeros(sizefft - len(A))))
    Bfilled = np.concatenate((B, np.zeros(sizefft - len(B))))
    out = ifft(fft(Afilled) * fft(Bfilled))
    return out[0:out_len]
```

```
92 def comprimir(x):
93     """Compresión de un pulso: convolución con el filtro apareado, recortada para alinear el
       pico con el rango."""
94     y = fastconv(x, matched_filter)
95     i0 = (len(matched_filter) - 1)//2
96     return y[i0:i0 + len(x)]
97
98
99 compressed = np.array([comprimir(p) for p in radar_signal])
```

Qué hace. `fastconv` (de la cátedra) convoluciona en frecuencia: rellena con ceros las dos señales hasta la potencia de 2 siguiente, multiplica sus FFT y antitransforma. `comprimir` aplica eso a un pulso y se queda con las 2000 muestras centrales; la última línea lo repite para los 100 pulsos.

Justificación teórica.

- Convolucionar en el tiempo es multiplicar en frecuencia: $y = x * h \Leftrightarrow Y(f) = X(f)\,H(f) = X(f)\,S^*(f)$. Con FFT el costo baja de N^2 a $N \log N$ operaciones.
- El relleno con ceros hasta al menos $N_x + N_h - 1$ muestras evita la convolución circular: sin él, el final de la salida se mezclaría con el principio.
- La convolución completa tiene $2000 + 2000 - 1 = 3999$ muestras. El filtro está centrado en $t = 0$ (el chirp se generó en $[-t_{max}/2,\ t_{max}/2]$), así que el pico de un eco aparece desplazado $(N_h - 1)/2$ muestras. Recortar desde ese índice deja cada pico en su rango (se verificó con un eco sintético a 10 km).
- Resultado: un eco que ocupaba $cT/2 = 1500$ m se concentra en un pico de $c/(2B) = 75$ m, con una ganancia de $BT = 20$ (13 dB). Por eso en la Figura `02` los blancos, ocultos en la señal cruda, aparecen como picos.

5. Ventana CFAR y umbral

```
104 def ventana_cfar():
105     w = np.concatenate((np.ones(N_REF//2), np.zeros(2*N_GAP + 1), np.ones(N_REF//2)))
106     return w/N_REF
107
108
109 H_CFAR = ventana_cfar()
110
111
112 def ruido_cfar(potencia):
113     """Potencia media de las celdas de referencia. En los bordes del vector hay menos celdas,
       por lo que se
114     normaliza por la cantidad de celdas realmente disponibles (sin esto el umbral cae en los
       extremos)."""
115     suma = np.convolve(potencia, H_CFAR*N_REF, mode='same')
116     celdas = np.convolve(np.ones_like(potencia), H_CFAR*N_REF, mode='same')
117     return suma/celdas
118
119
120 def umbral_cfar(potencia):
121     """Umbral CA-CFAR en amplitud: ganancia * sqrt(potencia media de las celdas de referencia)."
       ""
122     return np.sqrt(ALFA*ruido_cfar(potencia))
```

Qué hace.

- `ventana_cfar`: arma la ventana: 32 unos, 17 ceros (8 de guarda, la celda bajo prueba y 8 de guarda) y 32 unos, dividido por 64. Es la que se grafica en `03_ventana_cfar`.

- `ruido_cfar`: convoluciona la potencia con la ventana, lo que da para cada celda el promedio de sus 64 vecinas. La segunda convolución cuenta cuántas celdas hay realmente, para corregir los bordes del vector.
- `umbral_cfar`: multiplica ese ruido por α y pasa a amplitud con la raíz.

Justificación teórica. Un umbral fijo no sirve, porque el nivel de ruido y de clutter cambia con el rango, el clima o la dirección. El CA-CFAR (*cell averaging*) estima el ruido localmente:

$$\hat{P}[n] = \frac{1}{N} \sum_{k \in \text{referencia}} |y[n-k]|^2, \qquad \text{detección si } |y[n]|^2 > \alpha\, \hat{P}[n]$$

- Las **celdas de guarda** quedan fuera del promedio porque el propio blanco ocupa varias celdas (el pico comprimido y sus lóbulos, ± 110 m); si entraran, el blanco elevaría su propio umbral.
- Con ruido gaussiano complejo detectado en potencia, $\hat{P}$ promedia N variables exponenciales y la probabilidad de falsa alarma queda $P_{fa} = (1 + \alpha/N)^{-N}$, independiente del nivel de ruido (de ahí "tasa de falsa alarma constante"). Despejando:

$$\alpha = N\left(P_{fa}^{-1/N} - 1\right) = 64\left(10^{6/64} - 1\right) = 15{,}4 \;\Rightarrow\; \sqrt{\alpha} = 3{,}93$$

- **Elección de** N: con más celdas la estimación es mejor (menos pérdida CFAR), pero un blanco fuerte dentro de la ventana de otro le eleva el umbral y lo oculta (enmascaramiento). Con las 200 celdas del ejemplo de la cátedra se perdían blancos; con 64 el efecto se reduce.

6. Detección por cruces por cero

```
MIN_SEP = int(round(2*res_rango/rank_res))        # 10 celdas = 150 m: dentro de esto, lóbulos del mismo blanco


def detectar(amplitud, umbral, min_sep=MIN_SEP):
    """Detección por cruces por cero de (señal - umbral); devuelve el índice del máximo de cada cruce."""
    d = np.sign(amplitud - umbral)
    d[ranks < rank_min] = -1                        # celdas dentro del rango mínimo (blanqueo del receptor)
    cruces = np.diff(d)
    subidas = np.where(cruces > 0)[0] + 1
    bajadas = np.where(cruces < 0)[0] + 1
    picos = []
    for s in subidas:
        b = bajadas[bajadas > s]
        e = b[0] if len(b) else len(amplitud)
        picos.append(s + int(np.argmax(amplitud[s:e])))
    # fusiona picos separados por menos de min_sep celdas (lóbulos del mismo blanco)
    fusion = []
    for p in picos:
        if fusion and p - fusion[-1] < min_sep:
            if amplitud[p] > amplitud[fusion[-1]]:
                fusion[-1] = p
        else:
            fusion.append(p)
    return fusion, cruces
```

Qué hace. Resta el umbral a la señal y toma el signo ($+1$ arriba, -1 abajo). Fuerza -1 antes del rango mínimo. La diferencia entre muestras consecutivas vale $+2$ donde empieza una detección y -2 donde termina. Dentro de cada intervalo se queda con el máximo, y une picos a menos de 150 m.

Justificación teórica. Es lo que pide la consigna con `numpy.sign` y `numpy.diff`: los cruces por cero de $|y[n]| - V_T[n]$ marcan los bordes de cada zona sobre el umbral. El rango del blanco es el del máximo de esa zona, porque el filtro apareado concentra allí la energía. Los picos a menos de dos celdas de resolución (150 m) se unen porque corresponden al mismo eco y a sus lóbulos laterales.

7. Canceladores MTI simple, STI simple y MTI doble

```
153 mti_sc = compressed[1:] - compressed[:-1]                        # H(z) = 1 - z^-1
154 sti_sc = compressed[1:] + compressed[:-1]                        # H(z) = 1 + z^-1
155 mti_dc = compressed[2:] - 2*compressed[1:-1] + compressed[:-2]    # H(z) = 1 - 2 z^-1 + z^-2
```

Qué hace. Opera entre pulsos consecutivos, celda de rango por celda de rango. `compressed[1:]` son los pulsos 1 a 99 y `compressed[:-1]` los pulsos 0 a 98: restarlos es "cada pulso menos el anterior" para todas las distancias a la vez.

Justificación teórica. Un blanco a distancia R que se mueve con velocidad radial v cambia su fase entre pulsos en $\Delta\phi = 2\pi f_D\, PRI$, con $f_D = 2v/\lambda$. Los canceladores son filtros FIR en tiempo lento:

$$\begin{aligned} \text{MTI simple: } & H(z) = 1 - z^{-1}, & |H(f_D)| &= 2\,|\sin(\pi f_D/PRF)| \\ \text{STI simple: } & H(z) = 1 + z^{-1}, & |H(f_D)| &= 2\,|\cos(\pi f_D/PRF)| \\ \text{MTI doble: } & H(z) = (1 - z^{-1})^2, & |H(f_D)| &= 4\sin^2(\pi f_D/PRF) \end{aligned}$$

- **MTI**: pasaaltos en Doppler. Un eco fijo ($f_D = 0$) es igual en todos los pulsos y la resta lo anula: elimina el clutter y los blancos detenidos.
- **STI**: pasabajos. Refuerza lo fijo y anula los blancos con f_D cerca de $\pm PRF/2$, como el de 10,02 km de la señal 1 o el de 26,24 km de la señal 2.
- **MTI doble**: dos MTI en cascada. Su nulo en $f_D = 0$ es más ancho y profundo, mejor para clutter con algo de dispersión Doppler, pero atenúa más a los blancos lentos.
- Las respuestas son periódicas con período PRF: un blanco con $v = k\,\lambda\,PRF/2 = k \cdot 173$ m/s es cancelado como si estuviera quieto (**velocidad ciega**).

8. CFAR plot de cada cancelador

```
193 def cfar_plot(nombre, archivo, salida, pulsos, signos):
194     """Gráfico CFAR de un cancelador usando los pulsos indicados (momento de muestreo t0, t1,
        ...)."""
195     amp = np.abs(salida[0])
196     thr = umbral_cfar(amp**2)
197     det, cruces = detectar(amp, thr)
198     # Detección integrada (no coherente) sobre todos los momentos de muestreo
199     pot_int = np.mean(np.abs(salida)**2, axis=0)
200     amp_int = np.sqrt(pot_int)
201     det_int, _ = detectar(amp_int, umbral_cfar(pot_int))
```

```
226 r_mti, r_mti_int, amp_mti_int = cfar_plot('MTI simple cancelador', '04_cfar_mti_sc.png', mti_sc,
        [0, 1], '$x_1 - x_0$')
227 r_sti, r_sti_int, amp_sti_int = cfar_plot('STI simple cancelador', '05_cfar_sti_sc.png', sti_sc,
        [0, 1], '$x_1 + x_0$')
228 r_mdc, r_mdc_int, _ = cfar_plot('MTI doble cancelador', '06_cfar_mti_dc.png', mti_dc, [0, 1, 2],
        '$x_2 - 2x_1 + x_0$')
```

Qué hace. Para cada cancelador calcula el umbral y las detecciones sobre su primera salida, que usa los pulsos t_0 y t_1 (o t_0, t_1 y t_2 para el doble), como pide la consigna para el gráfico. Además

repite la detección sobre la potencia promediada de todas las salidas. El resto de la función arma los cuatro paneles de las figuras `04` a `06`: señales sin comprimir, comprimidas, salida con umbral y cruces por cero.

Justificación teórica. Detectar con un solo par de pulsos es lo que pide el gráfico, pero es sensible a la fluctuación del blanco y del ruido. Promediar $|y|^2$ sobre los 99 (o 98) pulsos es una **integración no coherente**: no usa la fase, pero reduce la varianza del ruido y da una lista de detecciones más estable. Se usa para confirmar blancos y para clasificar los fijos.

9. Filtrado Doppler

```
m = mti_sc.shape[0]
row = np.arange(m).reshape(-1, 1); col = np.arange(m).reshape(1, -1)
FD = np.exp(-1j*2*np.pi*row*col/m)                    # matriz de la DFT [m x m]
doppler = mti_sc.T @ FD                               # [rango x frecuencia Doppler]
doppler = fftshift(doppler, axes=1)/m
# Convención de la cátedra: velocidad positiva = el blanco se acerca (f_D = 2v/lambda > 0). En las señales
# simuladas, un blanco que se acerca cae en las frecuencias negativas de esta DFT, por lo que se invierte el
# eje Doppler (con m impar el eje es simétrico y la inversión es exacta).
doppler = doppler[:, ::-1]
fd = (np.arange(m) - m//2)*PRF/m                      # frecuencias Doppler [Hz]
vel = fd*wlen/2                                       # velocidades radiales [m/s], positiva = se acerca
```

Qué hace.

- Arma la matriz de la DFT de $m = 99$ puntos, $F_D[r, c] = e^{-j2\pi rc/m}$, como en el diagrama de la consigna.
- `mti_sc.T @ FD`: transpone la salida del MTI (cada fila pasa a ser una distancia con sus 99 valores en el tiempo) y la multiplica por la matriz. Eso es la DFT de cada fila: el espectro Doppler de cada celda de rango.
- `fftshift` lleva la frecuencia cero al centro, y `fd` y `vel` dan la frecuencia y la velocidad de cada columna.

Justificación teórica. Para un blanco en la celda n, la secuencia en tiempo lento es $x_p = A\, e^{j2\pi f_D\, p\, PRI}$: una exponencial compleja cuya frecuencia es la Doppler. La DFT la concentra en la columna k con $f_D = (k - m/2)\, PRF/m$, y la velocidad es

$$v_r = \frac{f_D\, \lambda}{2}, \qquad \Delta v = \frac{\lambda\, PRF}{2m} = 1{,}75 \text{ m/s}$$

- La DFT suma los 99 pulsos con su fase corregida: es una **integración coherente**, con una ganancia de hasta $10 \log_{10} 99 \approx 20$ dB.
- Blancos cercanos en rango pero con distinta velocidad quedan en columnas distintas. Así se recuperan el de 15,51 km (señal 1) y el de 19,82 km (señal 2), que el CFAR de los canceladores no separaba.
- Se usa la salida del MTI para que el clutter de velocidad cero ya esté atenuado.
- El signo sigue la convención de la cátedra: $v > 0$ es acercamiento ($f_D = 2v/\lambda > 0$) y $v < 0$ alejamiento. En las señales simuladas, un blanco que se acerca cae en las frecuencias negativas de la DFT, por eso el código invierte el eje Doppler con `doppler[:, ::-1]`; como $m = 99$ es impar, el eje es simétrico y la inversión es exacta.

9.1. Detección sobre el mapa rango-velocidad

```
doppler_w = fftshift((mti_sc*np.hamming(m)[:, None]).T @ FD, axes=1)[:, ::-1]/m   # mismo eje que doppler
P_rd = np.abs(doppler_w)**2
det_rd = np.zeros_like(P_rd, dtype=bool)
for k in range(m):
    det_rd[:, k] = P_rd[:, k] > ALFA*ruido_cfar(P_rd[:, k])
det_rd[ranks < rank_min, :] = False
```

Qué hace. Repite la DFT ponderando el tiempo lento con una ventana de Hamming y aplica el mismo CA-CFAR en rango para cada columna de velocidad.

Justificación teórica. La DFT con ventana rectangular tiene lóbulos laterales de -13 dB: alrededor de un blanco de 40 dB de SNR aparecerían detecciones falsas en otras velocidades. La ventana de Hamming los baja a -43 dB, a costa de un pico algo más ancho. La figura 07 (3D) se hace con la DFT sin ventana, como pide la consigna; la ventana solo se usa para detectar.

10. Clasificación de los blancos

```
confirmadas = r_mti + r_sti + r_mdc + r_mti_int + r_sti_int + r_mdc_int
moviles, descartes = [], []
for b in detecciones_rd:
    vecinos = [o for o in detecciones_rd if abs(o['_i'] - b['_i']) <= MIN_SEP and o['snr_db'] > b['snr_db']]
    if any(o['snr_db'] - b['snr_db'] > 20 for o in vecinos):
        descartes.append(dict(b, motivo='residuo de un blanco más fuerte')); continue
    confirmada = any(abs(ranks[b['_i']] - r) <= 2*res_rango for r in confirmadas)
    if not (confirmada or b['snr_db'] >= 20):
        descartes.append(dict(b, motivo='falsa alarma (débil y sin confirmar)')); continue
    if abs(b['velocidad_ms']) < PRF/m*wlen/2*1.5:
        continue                                          # velocidad nula: lo clasifica el criterio de blancos fijos
    moviles.append(b)
fijos = []
for r in r_sti_int:
    i = int(np.argmin(np.abs(ranks - r)))
    rel_db = 20*np.log10(amp_sti_int[i]/amp_mti_int[i])
    if rel_db > 10:
        fijos.append(dict(rango_km=round(r/1e3, 3), sti_sobre_mti_db=round(float(rel_db), 1)))
```

Qué hace. Junta las detecciones del mapa Doppler y de los canceladores y decide qué es cada una:

- **Residuo**: hay otra detección en la misma celda de rango más de 20 dB más fuerte. Se descarta.
- **Falsa alarma**: SNR menor que 20 dB y ningún cancelador la confirma a menos de 150 m. Se descarta.
- **Móvil**: lo que queda, con su velocidad del mapa Doppler.
- **Fijo**: detectado por el STI integrado, donde el STI supera al MTI en más de 10 dB.

Justificación teórica.

- Un blanco real fluctúa en amplitud de pulso a pulso (modelos de Swerling); esa modulación ensancha su espectro Doppler y deja energía en otras velocidades, sobre todo en cero. Esos residuos están mucho más abajo que el pico del blanco.
- Con $P_{fa} = 10^{-6}$ y unas 2000×99 celdas en el mapa se esperan del orden de 0,2 falsas alarmas por mapa; las tres detecciones débiles en 1,5, 6,2 y 13,7 km aparecen en todas las señales, lo que indica que comparten la realización del ruido. Por eso se exige confirmación.

- Un blanco fijo tiene $f_D = 0$: el STI lo duplica (+6 dB) y el MTI lo anula. Una diferencia de más de 10 dB entre ambos indica velocidad nula.

11. Gráficos que pide la consigna

Archivo	Contenido	Líneas
`01_senal_recibida`	Un momento de muestreo: real, imaginaria y módulo vs. rango	159–170
`02_compresion`	Módulo sin comprimir y comprimido vs. rango	172–180
`03_ventana_cfar`	Ventana CFAR con la cantidad de celdas	182–189
`04/05/06_cfar_*`	CFAR plot de MTI SC, STI SC y MTI DC	193–228
`07_doppler_3d`	Superficie 3D del producto Doppler (velocidad, rango)	300–310
`08_mapa_rango_velocidad`	Vista superior en dB con los blancos marcados	312–325